

Linear_Regression_3

Content

- * Recap
- * Importance of train test split
- * Feature importance and interpretation
- * Importance of standardization
- * Min max scaler
- * Case study: Predict mileage of car
Loading dataOne Hot encodingCreate dummy variablesHandling missing valuesData splittingFit linear model
- * Impact of outliers
- * Assumptions in Linear Regression
Assumption of LinearityFeatures are not multi-collinear
VIFErrors are normally distributedError terms must be independentHomoscedasticity: Error has constant variance
- * Polynomial regression
- * Occams Razor
- * Bias variance Tradeoff
- * Regularization

What did we learn in last lecture ?

- * We looked at how we define the functions
- * Understood the z score
- * Learnt the importance of standardization (Min - max Scaler)
- * We also digged into code and understood how to create LinearRegression object.
- * We understood what does `fit` function do.
- * We also understood about errors
- * At last, we also discussed why R^2 is important.

When working on ML model, what all things to keep in mind ?

Lets say we have a dataset (say 10 datapoints)

Importance of train test split

Recall that we talked about train and test data?

When we build a model, we dont take whole data but a portion of data and term it as training set.

We use the remaining set as test data.

For building model,

- we can choose first 8 rows/ datapoints as training set and
- remaining 2 rows will be available as test data.

In Real world,

when we do this train test split,

- It is not mandatory to take training data in any order.

When we have 10 records,

- and we are dividing it into 8 and 2.
- It means 80% - 20%

Now, these 2 test datapoints can be any two points.

- It need not be last two datapoints.

Notice that,

- In this case, we also have 80-20 split but in random way.

Generally,

- when we split data in train and test, it has to be complete random.
- i.e. theres no pattern in datapoints selection for train/test.

Question: What should be a good split ratio ?

Generally,

- * Youll find 80-20 split ratio.

But, if you have good enough data (say 10k datapoint)

- then we use 70-30 split i.e.
- Built model with 70% of datapoint and
- use rest 30% for test.

It depends on how much data you have with you.

- If you don't have sufficient amount of data with you, then
- 80-20 split can be preferred.

* Else, 70-30 split can be chosen.

We have a function/API which helps us in splitting the data and we'll explore it in today's lecture

Question: What do we do after splitting the data ?

* Now, we have the train data.

* From train data, we build a mathematical function (say, model)

Question: What do we do with this model ?

* Using this trained model,

We run this model on test data. We use the function on test data and get the predictions

After getting the predictions,

- we calculate the R^2 on Test data.

If this R^2 comes out to be good, then we can say this model is a reliable model.

- Because it's no use if we do the predictions on train data and calculate the R^2 (done in previous lecture)

In previous lecture, we used train data to build the model and we were using same to test the model.

- So, we'll definitely get a good R^2 on it.

To summarize:

- On train data, we'll build the model
- On test data, we'll test our model performance.

Feature importance and Interpretation

Say, we are predicting sales of a product (say, Scaler DSML course) based on TV ads spending (column A) and Social Media ads spending (column B).

Question: What will be the regression equation for this ?

Ans: $y = w_0 + w_1 A + w_2 B$

Lets say, the coef. for our model are as follows:

Equation: $y = 3 + 2TV + 3SM$

Question: What does this model tell us ?

It tells us that

- Say, we kept our TV budget as constant and increased our SM ads budget by 1 unit, itll increase/impact our sales by 3 units.

* If we keep our SM ads budget as constand and icrease the TV ads budget by 1 unit, itll increase sales by 2 units

What does this means?

* It means higher the value coef, better the value.

Now, suppose, our model is:

$$y = 3 + 2TV - 3SM$$

Question: What does this mean?

It means social media is impact our sales negatively.

- For example: We put up an ad which was percieved wrongfully and boycott trends start.

This is how we interpret the coef.

Importance of standardization

We had 2 features i.e. TV budget and SM budget.

Now,

- values of TV budget are written in exact figures whereas

* SM budget values have been factorized and are written for a million.

For example: when the value of SM is 3, it means 3 million.

When we built the model, the equation will be:

$$w_0 + w_1TV - w_2SM = \text{sales}$$

The values of TV budget are large whereas

- the values of SM budget are at max of 2 digit (due to fix of scale to million)

* While calculating the coef, itll drive model to pick small value for w_1 and increase the value of w_2 to balance out things.

Here, variation in units are going to create problem for us.

Question: How can we resolve this issue ?

=> Standardization / Normalization

We can do this using two methods:

- If we use z-score here, we'll bring everything to same units

* Another technique is min-max scaler.

Min max scaler

It brings all the values in between the range of 0 - 1

Question: What two values do we need to convert values to 0 - 1 range ?

Ans:

Min max scaling = $\frac{x_i - \min}{\max - \min}$

x_i ranges from (min, max)

* If we put $x_i = \min$, then

$\Rightarrow \frac{\min - \min}{\max - \min} = 0$

* If we put $x_i = \max$, then

$\Rightarrow \frac{\max - \min}{\max - \min} = 1$

Question: Based on this, what will be the issue with Min max scaling ?

Outlier can impact min max scaler.

* As outlier value will be treated as max value,

while scaling, it'll squeeze rest of the values to a very small range say 0 - 0.2. Most of the values will sit near min value.

Code walkthrough

Predict mileage using given attributes.

Code

```
import numpy as np
from sklearn.linear_model import LinearRegression
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
from sklearn.model_selection import train_test_split # Sklearn package's randomized data
splitting function
```

Load and review data

Code

```
!gdown 1RN3Fncv6sypLDHRjYDlQlKC_ATXC5yB6
```

Output

```
Downloading...
From: https://drive.google.com/uc?id=1RN3Fncv6sypLDHRjYDlQlKC_ATXC5yB6
To: /content/auto-mpg.csv
0% 0.00/18.1k [00:00<?, ?B/s]100% 18.1k/18.1k [00:00<00:00, 30.6MB/s]
```

Code

```
cData = pd.read_csv("auto-mpg.csv")
cData.shape
```

Output

```
(398, 9)
```

Code

```
# 8 variables:
#
# MPG (miles per gallon),
# cylinders,
# engine displacement (cu. inches),
# horsepower,
# vehicle weight (lbs.),
# time to accelerate from 0 to 60 mph (sec.),
# model year (modulo 100), and
# origin of car (1. American, 2. European,3. Asian).
#
# Also provided are the car labels (types)
# Missing data values are marked by series of question marks.
cData.head()
```

Output

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	year	\
0	18.0		8	307.0	130	3504	12.0		70
1	15.0		8	350.0	165	3693	11.5		70
2	18.0		8	318.0	150	3436	11.0		70
3	16.0		8	304.0	150	3433	12.0		70
4	17.0		8	302.0	140	3449	10.5		70
origin			car name						
0	1	chevrolet chevelle malibu							
1	1	buick skylark 320							
2	1	plymouth satellite							
3	1	amc rebel sst							
4	1	ford torino							

Here, origin can have 3 distinct values:

- 1 => American
- 2 => European
- 3 => Asian

Question: Do we need car name to predict the mileage ?

Ans: No. So, let's drop the carname.

Let's also replace the origin value with their corresponding location

Code

```
#dropping/ignoring car_name
cData = cData.drop('car name', axis=1)
# Also replacing the categorical var with actual values
cData['origin'] = cData['origin'].replace({1: 'American', 2: 'European', 3: 'Asian'})
cData.head()
```


Output

```
mpg  cylinders  displacement horsepower   weight  acceleration  model year  \
0  18.0         8         307.0      130     3504         12.0      70
1  15.0         8         350.0      165     3693         11.5      70
2  18.0         8         318.0      150     3436         11.0      70
3  16.0         8         304.0      150     3433         12.0      70
4  17.0         8         302.0      140     3449         10.5      70
origin
0  American
1  American
2  American
3  American
4  American
```

Recall that we cant use any categorical data into our regression equation.

- We need to have our data in numbers

How can we convert categorical to numerical data ?

There are various techniques for encoding/ converting categorical to numerical data.

In this example, well use a simple technique called One Hot Encoding

One Hot encoding

Say, we have two columns

- Age
- Origin

Lets say, we assigned

- America = 1
- India = 2
- Australia = 3

Question: Will this work ?

There's an issue with this approach.

* Since, we are assigning 3 to Australia
It means we are giving more weightage to it.

Our equation is $y = mx + c$

* When we put $x = 3$ in it, it means we are giving more weightage to it; which isn't fair.

We solve this using One Hot Encoding

* All the unique values of origin will become separate columns
i.e. We'll have a separate column for India, America and Australia

In original column, we have origin as America for first row.

* So, we'll populate the America column as 1. In rest of the column, we'll put 0.

Similarly,

- we have India as origin for second row.

So, we'll put India as 1 and rest two as zero

Using this technique, we are not giving any advantage to specific value.

Question: Say, you know the values for America and India column. Can you figure out values for Australia column ?

Ans: Yes. If we know values for two out of 3 columns, we can automatically find the value for 3rd column

* This makes one of the columns as redundant.

* and we can remove one of the columns

So, we'll simply remove one column (Australia)

Let's apply it using code

Create Dummy Variables

Values like America cannot be read into an equation. Using substitutes like 1 for America, 2 for Europe and 3 for Asia would end up implying that European cars fall exactly half way between American and Asian cars! We don't want to impose such an baseless assumption!

So we create 3 simple true or false columns with titles equivalent to Is this car America?, Is this car European? and Is this car Asian?. These will be used as independent variables without imposing any kind of ordering between the three regions.

Code

```
cData = pd.get_dummies(cData, columns=['origin'])
cData.head()
```

Output

```
mpg  cylinders  displacement  horsepower  weight  acceleration  model year  \
0   18.0         8           307.0         130     3504           12.0     70
1   15.0         8           350.0         165     3693           11.5     70
2   18.0         8           318.0         150     3436           11.0     70
3   16.0         8           304.0         150     3433           12.0     70
4   17.0         8           302.0         140     3449           10.5     70
origin_American  origin_asia  origin_europe
0              True         False         False
1              True         False         False
2              True         False         False
3              True         False         False
4              True         False         False
```

Dealing with Missing Values

Code

```
#A quick summary of the data columns
cData.describe()
```

Output

```
mpg   cylinders  displacement      weight  acceleration  \
count 398.000000 398.000000    398.000000    398.000000    398.000000
mean  23.514573  5.454774    193.425879   2970.424623    15.568090
std    7.815984  1.701004    104.269838    846.841774     2.757689
min    9.000000  3.000000     68.000000   1613.000000     8.000000
25%   17.500000  4.000000    104.250000   2223.750000    13.825000
50%   23.000000  4.000000    148.500000   2803.500000    15.500000
75%   29.000000  8.000000    262.000000   3608.000000    17.175000
max   46.600000  8.000000    455.000000   5140.000000    24.800000
model year
count 398.000000
mean  76.010050
std    3.697627
min   70.000000
25%   73.000000
50%   76.000000
75%   79.000000
max   82.000000
```

Datatypes of column

Code

```
# hp is missing cause it does not seem to be recognized as a numerical column!
cData.dtypes
```

Output

```
mpg          float64
cylinders     int64
displacement  float64
horsepower    object
weight        int64
acceleration  float64
model_year    int64
origin_American  bool
origin_asia    bool
origin_europe  bool
dtype: object
```

Notice that

- dtype of horsepower is

Code

```
# isdigit()? on 'horsepower'
hpIsDigit = pd.DataFrame(cData.horsepower.str.isdigit()) # if the string is made of digits
store True else False
#print isDigit = False!
cData[hpIsDigit['horsepower'] == False] # from temp take only those rows where hp has false
```

Output

mpg	cylinders	displacement	horsepower	weight	acceleration	\
32	25.0	4	98.0	?	2046	19.0
126	21.0	6	200.0	?	2875	17.0
330	40.9	4	85.0	?	1835	17.3
336	23.6	4	140.0	?	2905	14.3
354	34.5	4	100.0	?	2320	15.8
374	23.0	4	151.0	?	3035	20.5
model	year	origin_American	origin_asia	origin_europe		
32	71	True	False	False		
126	74	True	False	False		
330	80	False	False	True		
336	80	True	False	False		
354	81	False	False	True		
374	82	True	False	False		

we have some missing values in columns.

We'll convert it to NaNs

Code

```
# Missing values have a '?'
# Replace missing values with NaN
cData = cData.replace('?', np.nan)
cData[hpIsDigit['horsepower'] == False]
```

Output

mpg	cylinders	displacement	horsepower	weight	acceleration	\
32	25.0	4	98.0	NaN	2046	19.0
126	21.0	6	200.0	NaN	2875	17.0
330	40.9	4	85.0	NaN	1835	17.3
336	23.6	4	140.0	NaN	2905	14.3
354	34.5	4	100.0	NaN	2320	15.8
374	23.0	4	151.0	NaN	3035	20.5
model year	origin_American	origin_asia	origin_europe			
32	71	True	False	False		
126	74	True	False	False		
330	80	False	False	True		
336	80	True	False	False		
354	81	False	False	True		
374	82	True	False	False		

Code

```
cData['horsepower'] = cData['horsepower'].astype(float)
```

Handling missing values

There are various ways to handle missing values.

- Drop the rows
- replace missing values with median values etc.

Out of 398 rows, 6 have NAN in the hp column.

We could drop those 6 rows - which might not be a good idea under all situations

Instead of dropping the rows, lets replace the missing values with median value.

Code

```
cData.median()
```

Output

```
mpg                23.0
cylinders           4.0
displacement       148.5
horsepower          93.5
weight            2803.5
acceleration        15.5
model_year          76.0
origin_American     1.0
origin_asia         0.0
origin_europe       0.0
dtype: float64
```

Code

```
# replace the missing values with median value.
# Note, we do not need to specify the column names below
# every column's missing value is replaced with that column's median respectively (axis =0
means columnwise)
#cData = cData.fillna(cData.median())
medianFiller = lambda x: x.fillna(x.median())
cData = cData.apply(medianFiller,axis=0)
cData['horsepower'] = cData['horsepower'].astype('float64') # converting the hp column from
object / string type to float
```


Split Data

Before splitting, we'll also drop one of the dummy columns (Europe) as it is redundant

Code

```
# let's build our linear model
# independent variables
X = cData.drop(['mpg', 'origin_europe'], axis=1)
# the dependent variable
y = cData[['mpg']]
```

Code

```
# Split X and y into training and test set in 70:30 ratio
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=1)
```

Fit Linear Model

Question: What should we pass in fit function ?

We'll pass to fit function

Code

```
regression_model = LinearRegression()
regression_model.fit(X_train, y_train)
```

Output

```
LinearRegression()
```

Question: How can we get the coef./ weights ?

Ans: After training the model, we can access the coef. using `coef_` property

Code

```
for idx, col_name in enumerate(X_train.columns):  
    print("The coefficient for {} is {}".format(col_name, regression_model.coef_[0][idx]))
```

Output

```
The coefficient for cylinders is -0.39480796616482383  
The coefficient for displacement is 0.02894551076548718  
The coefficient for horsepower is -0.021752207723546777  
The coefficient for weight is -0.007352032065147351  
The coefficient for acceleration is 0.061919366007618756  
The coefficient for model year is 0.836933891764499  
The coefficient for origin_American is -3.001283000918514  
The coefficient for origin_asia is -0.6060179643247368
```

Code

```
intercept = regression_model.intercept_[0]  
print("The intercept for our model is {}".format(intercept))
```

Output

The intercept for our model is -18.283451116372046

Question: How can we get R^2 score ?

Ans: Using

Training R^2 score

Code

```
regression_model.score(X_train, y_train)
```

Output

0.8141025501610559

Test R^2 score

Code

```
#out of sample score ( $R^2$ )  
regression_model.score(X_test, y_test)
```

Output

0.8433135132808833

Impact of outliers

Question: Will outliers impact the model (regression line) or not ?

When outliers are not present, you get a good line of best fit.

* When there is an outlier present, the line of best fit or regression line will get pulled towards it.

Why regression line is pulled towards outlier?

* The equation is such that it'll try to pull wherever the outlier is sitting so as to reduce the distance.

* The closer the regression line is to outlier, less the error.

Make sure to remove the outliers before you start working on dataset.

* Click on the link to try yourself: <https://observablehq.com/@yizhe-ang/interactive-visualization-of-linear-regression>

Problems with R-squared, Adjusted R-squared

Lets implement it for our earlier model and test data.

Code

```
y_hat = regression_model.predict(X_test)
Adj_R = 1 - (1-regression_model.score(X_test,
y_test)*(len(y_test)-1)/(len(y_test)-X_test.shape[1]-1))
print("Adjusted R-squared:", Adj_R )
```

Output

```
Adjusted R-squared: 0.9040928655894155
```

****Introduction to StatsModels****

* What is StatsModels? StatsModels is a Python module that provides classes and functions for the estimation of statistical models. Offers extensive support for conducting statistical tests and data exploration. Provides detailed outputs with comprehensive statistical information, unlike Scikit-Learn which focuses more on prediction.

* Why Use StatsModels for Linear Regression? Detailed Statistical Output:

Includes p-values, confidence intervals, R-squared values, and more. Helps in interpreting the significance of each predictor. Diagnostic Tools:

Offers functions to test the assumptions of linear regression. Facilitates residual analysis and detection of multicollinearity.

Lets revisit our Cars42 dataset for this explanation

Code

```
import statsmodels.api as sm
```

Code

```
!gdown 1bwRmKkPwmLKiqOgQ_LnKH0Vsc3mJKmVR
```

Output

```
Downloading...
From: https://drive.google.com/uc?id=1bwRmKkPwmLKiqOgQ_LnKH0Vsc3mJKmVR
To: /content/cars24-car-price-cleaned.csv
0% 0.00/1.69M [00:00<?, ?B/s]100% 1.69M/1.69M [00:00<00:00, 17.8MB/s]
```

Code

```
df = pd.read_csv('cars24-car-price-cleaned.csv')
df['make'] = df.groupby('make')['selling_price'].transform('mean')
df['model'] = df.groupby('model')['selling_price'].transform('mean')
df.head()
```

Output

```
selling_price  year  km_driven  mileage  engine  max_power  age  \
0            1.20  2012.0    120000    19.70    796.0    46.30  11.0
1            5.50  2016.0     20000    18.90   1197.0    82.00   7.0
2            2.15  2010.0     60000    17.00   1197.0    80.00  13.0
3            2.26  2012.0     37000    20.92    998.0    67.10  11.0
4            5.70  2015.0     30000    22.77   1498.0    98.59   8.0
make          model  Individual  Trustmark  Dealer  Diesel  Electric  LPG  \
0  4.684721  1.180000           1           0           0           0  0
1  5.458819  4.818750           1           0           0           0  0
2  5.458819  3.394000           1           0           0           0  0
3  4.684721  2.242676           1           0           0           0  0
4  5.858258  6.777576           0           0           1           0  0
Petrol  Manual  5  >5
0        1        1  1  0
1        1        1  1  0
2        1        1  1  0
3        1        1  1  0
4        0        1  1  0
```

The above is cleaned dataset made using the concepts learnt in the previous class

Code

```
df_train, df_test = train_test_split(df, test_size=0.3, random_state=1)
```

Scaling

Code

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df_train = pd.DataFrame(scaler.fit_transform(df_train), columns=df.columns)
df_test = pd.DataFrame(scaler.transform(df_test), columns=df.columns)
```

Code

```
y_train = df_train['selling_price']
X_train = df_train.drop('selling_price', axis=1)
```

Code

```
y_test = df_test['selling_price']
X_test = df_test.drop('selling_price', axis=1)
```

Now, lets train our Statsmodel on the data.

Code

```
y_train = np.array(y_train)
```

Code

```
X_sm = sm.add_constant(X_train) # Statmodels default is without intercept, to add intercept  
we need to add constant.  
model = sm.OLS(y_train, X_sm)  
results = model.fit()  
# Print the summary statistics of the model  
print(results.summary())
```

Output

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          0.946
Model:                  OLS    Adj. R-squared:       0.946
Method:                 Least Squares    F-statistic:       1.512e+04
Date:                   Thu, 09 Oct 2025    Prob (F-statistic): 0.00
Time:                   15:26:14    Log-Likelihood:    20675.
No. Observations:      13874    AIC:               -4.132e+04
Df Residuals:          13857    BIC:               -4.119e+04
Df Model:              16
Covariance Type:       nonrobust
=====

```

coef	std err	t	P> t	[0.025	0.975]
const	-4.619e+11	1.16e+11	-3.968	0.000	-6.9e+11 -2.34e+11
year	4.619e+11	1.16e+11	3.968	0.000	2.34e+11 6.9e+11
km_driven	-0.2530	0.035	-7.185	0.000	-0.322 -0.184
mileage	-0.2304	0.022	-10.445	0.000	-0.274 -0.187
engine	0.0740	0.015	4.960	0.000	0.045 0.103
max_power	0.0467	0.014	3.243	0.001	0.018 0.075
age	4.619e+11	1.16e+11	3.968	0.000	2.34e+11 6.9e+11
make	0.0632	0.005	13.992	0.000	0.054 0.072
model	0.8594	0.005	169.919	0.000	0.849 0.869
Individual	-0.0071	0.001	-7.145	0.000	-0.009 -0.005
Trustmark Dealer	-0.0070	0.005	-1.443	0.149	-0.016 0.002
Diesel	0.0067	0.004	1.707	0.088	-0.001 0.014
Electric	0.1287	0.024	5.306	0.000	0.081 0.176
LPG	0.0157	0.009	1.743	0.081	-0.002 0.033
Petrol	-0.0067	0.004	-1.658	0.097	-0.015 0.001
Manual	-0.0039	0.002	-2.496	0.013	-0.007 -0.001
5	-0.0161	0.004	-3.760	0.000	-0.025 -0.008
>5	-0.0235	0.005	-5.081	0.000	-0.033 -0.014

```

=====
Omnibus:                2586.081    Durbin-Watson:          1.980
Prob(Omnibus):          0.000    Jarque-Bera (JB):       56705.926
Skew:                   0.287    Prob(JB):               0.00
Kurtosis:               12.888    Cond. No.:               8.61e+14
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 7.32e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Lets look at few of the variables in this table:

- * Dep. Variable: This column displays the name of the dependent variable being predicted in the regression.
- * Model: It provides a concise representation of the model type and method used, such as OLS (Ordinary Least Squares).
- * R-squared: Represents the coefficient of determination (R-squared) value.
- * Adj. R-squared: This is the adjusted R-squared value, which accounts for the number of predictors in the model and adjusts the R-squared accordingly.
- * Coefficients Table:coef: Estimated coefficients for each predictor.std err: Standard error of the coefficients.t: t-statistic for testing the hypothesis that the coefficient is different from zero.**P>t**: p-value corresponding to the t-statistic.[0.025 0.975]: 95% confidence interval for the coefficients.

We will see more uses of statsmodel library as we continue with todays lecture.

Assumptions in Linear Regression

1. Assumption of Linearity

- * Relation between the any independent and dependent variables must be linear.
- * if your relationships are not linear, we do some kind of fix to handle the non linear relationship using polynomial regression (will see later)

2. Features are not multi-collinear

What does multi-collinearity mean ?

Lets say you have 4 independent features x_1, x_2, x_3, x_4 and a dependent variable y

We want the relation between independent and dependent variable as : $y = f(x)$

Question: But, should there be a dependency among the independent variables ?

Ans: No

This could be a problem if theres a dependency between two or among the independent variables

The dependency could look something like:

The above equation is basically saying:

- the values of x_2 are dependent on x_3 and x_4 .

and this is called as Multicollinearity.



How to fix the Multicollinearity

We fix the Multicollinearity using VIF (Variance Inflation Factor)

It fixes the multicollinearity using a very smart way.

Lets say we have 4 columns x_1, x_2, x_3, x_4

* First, itll assume x_1 is dependent columns and x_2, x_3, x_4 are indepedent columns

* Now, itll create/build a model on it and checks the R^2 value

Question: If the R^2 value is high, does this mean x_1 have some dependency coming among these columns ?

Ans: Yes

Next,

Well again build a model with x_2 as dependent variable and x_1, x_3, x_4 as independent variables

* Well check the R^2 value for it

* And get to know whether the x_2 is dependent on these variable or not based on the value of R^2

This way,

- it internally keeps on computing R^2 for all the columns across and looks for columns where R^2 is higher

- If the R^2 is higher, then those are columns which are creating high dependency

Formula for VIF



Rule of thumb for VIF

* It can be inferred that if $VIF_j > 10$, that feature is more collinear

* Rule of thumb:

if $VIF_j > 10$: Very High multicollinearity
if $5 \leq VIF_j < 10$: High multicollinearity
if $VIF_j < 5$: we can use that feature



Code

```
# Statsmodels implementation of Linear regression
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
X = df[df.columns.drop('selling_price')]
y = df["selling_price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
scaler = StandardScaler()
X_tr_scaled = scaler.fit_transform(X_train)
X_sm = sm.add_constant(X_tr_scaled) #Statsmodels default is without intercept, to add
intercept we need to add constant
sm_model = sm.OLS(y_train, X_sm).fit()
print(sm_model.summary())
```

Output

OLS Regression Results

```
=====
Dep. Variable:      selling_price    R-squared:                0.945
Model:              OLS              Adj. R-squared:           0.945
Method:             Least Squares    F-statistic:             1.701e+04
Date:               Thu, 09 Oct 2025  Prob (F-statistic):        0.00
Time:               15:26:15          Log-Likelihood:          -24523.
No. Observations:   15856            AIC:                     4.908e+04
Df Residuals:       15839            BIC:                     4.921e+04
Df Model:           16
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	6.5834		0.009	729.235	0.000	6.566 6.601
x1	0.2907		0.006	46.479	0.000	0.278 0.303
x2	-0.0809		0.010	-8.150	0.000	-0.100 -0.061
x3	-0.1924		0.016	-11.846	0.000	-0.224 -0.161
x4	0.1031		0.023	4.547	0.000	0.059 0.147
x5	0.0832		0.020	4.114	0.000	0.044 0.123
x6	-0.2907		0.006	-46.479	0.000	-0.303 -0.278
x7	0.2500		0.016	15.187	0.000	0.218 0.282
x8	4.0281		0.022	179.756	0.000	3.984 4.072
x9	-0.0687		0.009	-7.290	0.000	-0.087 -0.050
x10	-0.0124		0.009	-1.359	0.174	-0.030 0.005
x11	0.0573		0.038	1.526	0.127	-0.016 0.131
x12	0.0623		0.010	6.222	0.000	0.043 0.082
x13	0.0168		0.010	1.658	0.097	-0.003 0.037
x14	-0.0868		0.039	-2.249	0.025	-0.162 -0.011
x15	-0.0363		0.012	-3.007	0.003	-0.060 -0.013
x16	-0.1093		0.031	-3.532	0.000	-0.170 -0.049
x17	-0.1747		0.033	-5.361	0.000	-0.239 -0.111

```
=====
Omnibus:              3218.068    Durbin-Watson:              2.002
Prob(Omnibus):         0.000      Jarque-Bera (JB):           80809.770
Skew:                  0.347       Prob(JB):                   0.00
Kurtosis:              14.038      Cond. No.                   1.47e+15
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 3.41e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Multicollinearity and implementation

- * When one or more input variables exhibit strong correlation
- * One variable is enough to be part of the model.

Code

```
# VIF
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

Code

```
vif = pd.DataFrame()
X_t = pd.DataFrame(X_tr_scaled, columns=X_train.columns)
vif['Features'] = X_t.columns
vif['VIF'] = [variance_inflation_factor(X_t.values, i) for i in range(X_t.shape[1])]
vif['VIF'] = round(vif['VIF'], 2)
vif = vif.sort_values(by = "VIF", ascending = False)
vif
```

Error Output

```
/usr/local/lib/python3.12/dist-packages/statsmodels/stats/outliers_influence.py:197:
RuntimeWarning: divide by zero encountered in scalar divide
```

Output

Features	VIF	
0	year	inf
5	age	inf
13	Petrol	18.29
10	Diesel	17.29
16	>5	13.04
15	5	11.75
3	engine	6.30
7	model	6.16
4	max_power	5.02
6	make	3.33
2	mileage	3.24
14	Manual	1.79
12	LPG	1.25
11	Electric	1.23
1	km_driven	1.21
8	Individual	1.09
9	Trustmark Dealer	1.02

Note : VIF values tends to be infinity when there is a perfect correlation between the variables

Removing the variable with highest VIF

Code

```
cols2 = vif["Features"][1:].values
X2 = pd.DataFrame(X_tr_scaled, columns=X_train.columns)[cols2]
X2_sm = sm.add_constant(X2) #Statmodels default is without intercept, to add intercept we
need to add constant
sm_model = sm.OLS(list(y_train), X2_sm).fit()
print(sm_model.summary())
```

Output

OLS Regression Results

```
=====
Dep. Variable:          y      R-squared:          0.945
Model:                  OLS    Adj. R-squared:       0.945
Method:                 Least Squares    F-statistic:      1.701e+04
Date:                   Thu, 09 Oct 2025    Prob (F-statistic): 0.00
Time:                   15:26:17    Log-Likelihood:    -24523.
No. Observations:       15856    AIC:               4.908e+04
Df Residuals:           15839    BIC:               4.921e+04
Df Model:                16
Covariance Type:        nonrobust
=====
```

```
=====
coef      std err          t      P>|t|      [0.025      0.975]
-----
const                6.5834      0.009    729.235      0.000      6.566      6.601
age                 -0.5814      0.013   -46.479      0.000     -0.606     -0.557
Petrol              -0.0868      0.039    -2.249      0.025     -0.162     -0.011
Diesel               0.0573      0.038     1.526      0.127     -0.016      0.131
>5                  -0.1747      0.033    -5.361      0.000     -0.239     -0.111
5                   -0.1093      0.031    -3.532      0.000     -0.170     -0.049
engine               0.1031      0.023     4.547      0.000      0.059      0.147
model                4.0281      0.022   179.756      0.000      3.984      4.072
max_power            0.0832      0.020     4.114      0.000      0.044      0.123
make                 0.2500      0.016    15.187      0.000      0.218      0.282
mileage             -0.1924      0.016   -11.846      0.000     -0.224     -0.161
Manual              -0.0363      0.012    -3.007      0.003     -0.060     -0.013
LPG                  0.0168      0.010     1.658      0.097     -0.003      0.037
Electric             0.0623      0.010     6.222      0.000      0.043      0.082
km_driven           -0.0809      0.010    -8.150      0.000     -0.100     -0.061
Individual           -0.0687      0.009    -7.290      0.000     -0.087     -0.050
Trustmark Dealer    -0.0124      0.009    -1.359      0.174     -0.030      0.005
=====
```

```
Omnibus:              3218.068    Durbin-Watson:          2.002
Prob(Omnibus):         0.000    Jarque-Bera (JB):       80809.770
Skew:                   0.347    Prob(JB):                0.00
Kurtosis:              14.038    Cond. No.                12.7
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

As we see, there's no drop in R-sq. and adj. R-sq. value even after removing the variable

Lets now keep on removing features till we

- either have a feature with $vif < 5$

or

- have the models adj. r^2 score go below 0.85

Code

```
vif_thr = 5
r2_thr = 0.85
i = 1
feats_removed = ['year']
while True:
    vif = pd.DataFrame()
    X_t = pd.DataFrame(X_tr_scaled, columns=X_train.columns)[cols2]
    vif['Features'] = cols2
    vif['VIF'] = [variance_inflation_factor(X_t.values, i) for i in range(X_t.shape[1])]
    vif['VIF'] = round(vif['VIF'], 2)
    vif = vif.sort_values(by = "VIF", ascending = False)
    cols2 = vif["Features"][1:].values
    X2 = pd.DataFrame(X_tr_scaled, columns=X_train.columns)[cols2]
    X2_sm = sm.add_constant(X2) #Statmodels default is without intercept, to add intercept we
    need to add constant
    sm_model = sm.OLS(list(y_train), X2_sm).fit()
    if (vif.iloc[0]['VIF'] < vif_thr) or (sm_model.rsquared_adj < r2_thr):
        print('Reached threshold')
        print('Highest vif:', vif.iloc[0])
        print('Current adj.R2', sm_model.rsquared_adj)
        print('Features removed:', i)
        print('List of features removed:', feats_removed)
        break
    feats_removed.append(vif.iloc[0]['Features'])
    i += 1
```

Output

```
Reached threshold
Highest vif: Features      model
VIF          6.14
Name: 2, dtype: object
Current adj.R2 0.8322522946246915
Features removed: 3
List of features removed: ['year', 'Petrol', '>5']
```

Code

```
print(vif)
print(sm_model.summary())
```

Output

Features	VIF
2	model 6.14
1	engine 6.12
3	max_power 4.85
4	make 3.29
5	mileage 2.86
6	Diesel 2.14
7	age 1.87
8	Manual 1.78
0	5 1.70
9	Electric 1.22
10	km_driven 1.21
11	Individual 1.09
12	Trustmark Dealer 1.02
13	LPG 1.01

OLS Regression Results

Dep. Variable:	y	R-squared:	0.832
Model:	OLS	Adj. R-squared:	0.832
Method:	Least Squares	F-statistic:	6052.
Date:	Thu, 09 Oct 2025	Prob (F-statistic):	0.00
Time:	15:26:25	Log-Likelihood:	-33356.
No. Observations:	15856	AIC:	6.674e+04
Df Residuals:	15842	BIC:	6.685e+04
Df Model:	13		
Covariance Type:	nonrobust		

coef	std err	t	P> t	[0.025	0.975]
const		6.5834	0.016	417.802	0.000 6.553 6.614
engine		0.1661	0.039	4.260	0.000 0.090 0.242
max_power		1.4122	0.032	43.747	0.000 1.349 1.475
make		1.7818	0.024	72.860	0.000 1.734 1.830
mileage		-0.4077	0.027	-15.354	0.000 -0.460 -0.356
Diesel		0.6839	0.022	30.547	0.000 0.640 0.728
age		-1.7934	0.018	-99.502	0.000 -1.829 -1.758
Manual		-0.6359	0.020	-31.481	0.000 -0.675 -0.596
5		-0.1577	0.020	-7.717	0.000 -0.198 -0.118
Electric		0.1570	0.017	9.024	0.000 0.123 0.191
km_driven		-0.1660	0.017	-9.602	0.000 -0.200 -0.132
Individual		-0.1471	0.016	-8.966	0.000 -0.179 -0.115
Trustmark Dealer		-0.0305	0.016	-1.917	0.055 -0.062 0.001
LPG		0.0574	0.016	3.621	0.000 0.026 0.088

Omnibus:	1857.357	Durbin-Watson:	1.982
----------	----------	----------------	-------

[Output continues below...]

```
Prob(Omnibus):          0.000   Jarque-Bera (JB):          9261.740
Skew:                  0.466   Prob(JB):              0.00
Kurtosis:              6.626   Cond. No.           5.75
=====
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

3. Errors are normally distributed

If you get all your errors and plot them up

- you'll notice that it'll create a bell shaped curve
- i.e. errors are normally distributed

The third assumption of Linear Regression is that the residuals should follow a normal distribution.

* If you plot the error distribution and it is not normal, then it means there are outliers present in data.

4. Error terms must be independent

* If we plot a scatter plot between the target variable and the error term, then

* there shouldn't exist a pattern in the error

Usually occurs in time series data.

* We'll talk about it in detail once we learn time series

Lets See how is the error distribution for Car24

Code

```
X_sm = sm.add_constant(X_tr_scaled)
sm_model = sm.OLS(y_train, X_sm).fit()
```

Code

```
y_train.shape
```

Output

```
(15856, )
```

Code

```
Y_hat = sm_model.predict(X_sm)  
errors = Y_hat - y_train
```

Code

```
Y_hat.shape
```

Output

```
(15856, )
```

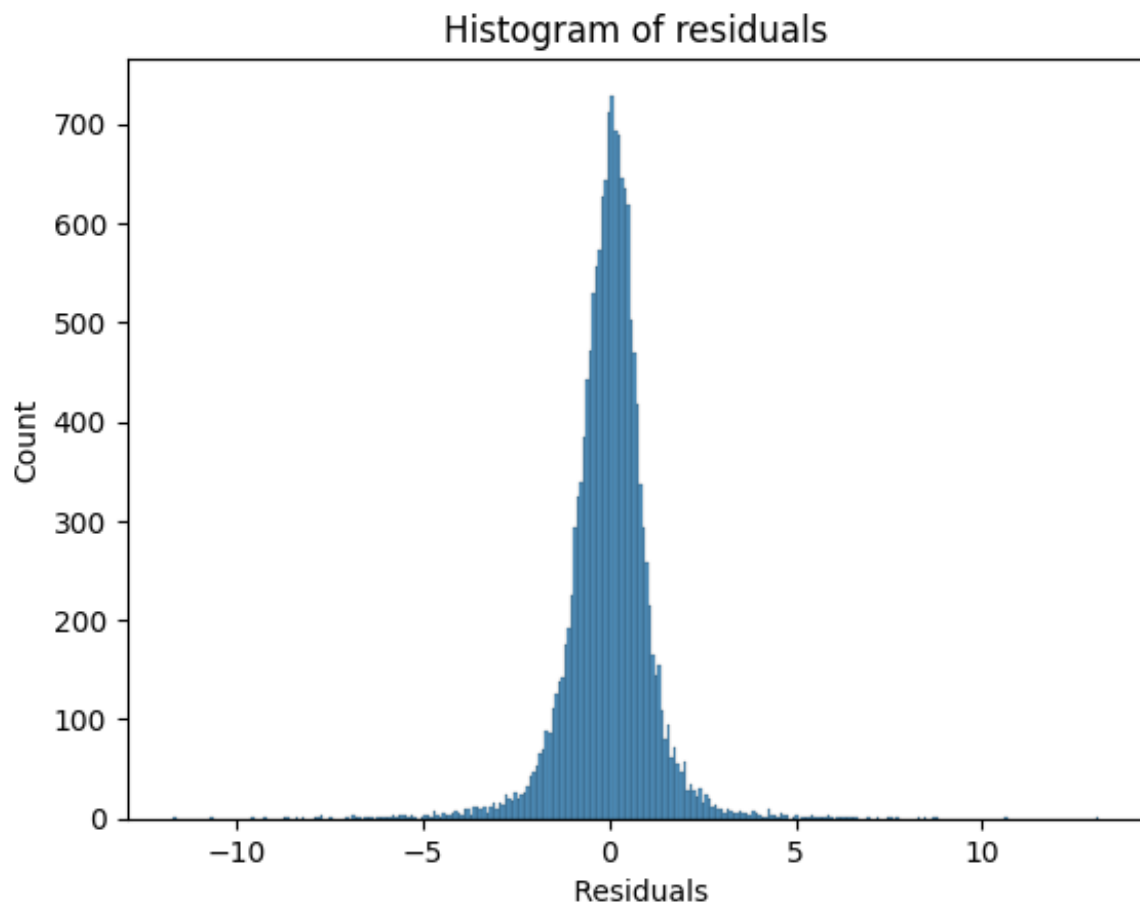
Code

```
import seaborn as sns  
sns.histplot(errors)  
plt.xlabel(" Residuals")  
plt.title("Histogram of residuals")
```

Output

```
Text(0.5, 1.0, 'Histogram of residuals')
```

Output Image



Code

```
errors
```

Output

```
3443    -0.193651
16090    0.124025
11070    0.404339
19214   -0.389842
17843   -0.006050
...
1099    -1.362943
18898   -0.487388
11798    0.450178
6637    -1.146986
2575     1.361600
Name: selling_price, Length: 15856, dtype: float64
```

This looks like a normal distribution, with a high mean and low S.D.

Lets do a normality test to check if the distribution is actually normal.

Recall:

- You must have studied Shapiro Wilk test in statistics module.

Code

```
from scipy import stats
res = stats.shapiro(errors)
res.statistic
```

Error Output

```
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: UserWarning:
scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate. Current N is 15856.
```

Output

```
np.float64(0.8901716755445348)
```

Closer the value to 1, more is the normality.

In this case, a value of 0.85 denotes a high level of normality for the error distribuiton

* If the distribution of errors is gaussian distribution then the assumption of error being normally distributed is being sastified (sanity check)

5. Homoscedasticity: Error has constant variance.

Homoscedasticity means to be of The same Variance, the error terms will be the same and of very little variance.

* A scatterplot of residuals versus predicted values is good way to check for homoscedasticity.

* You should get a graph like the left graph above.

Lets implement heteroskedasticity in Cars24

Code

```
Y_hat = sm_model.predict(X_sm)
errors = Y_hat - y_train
```

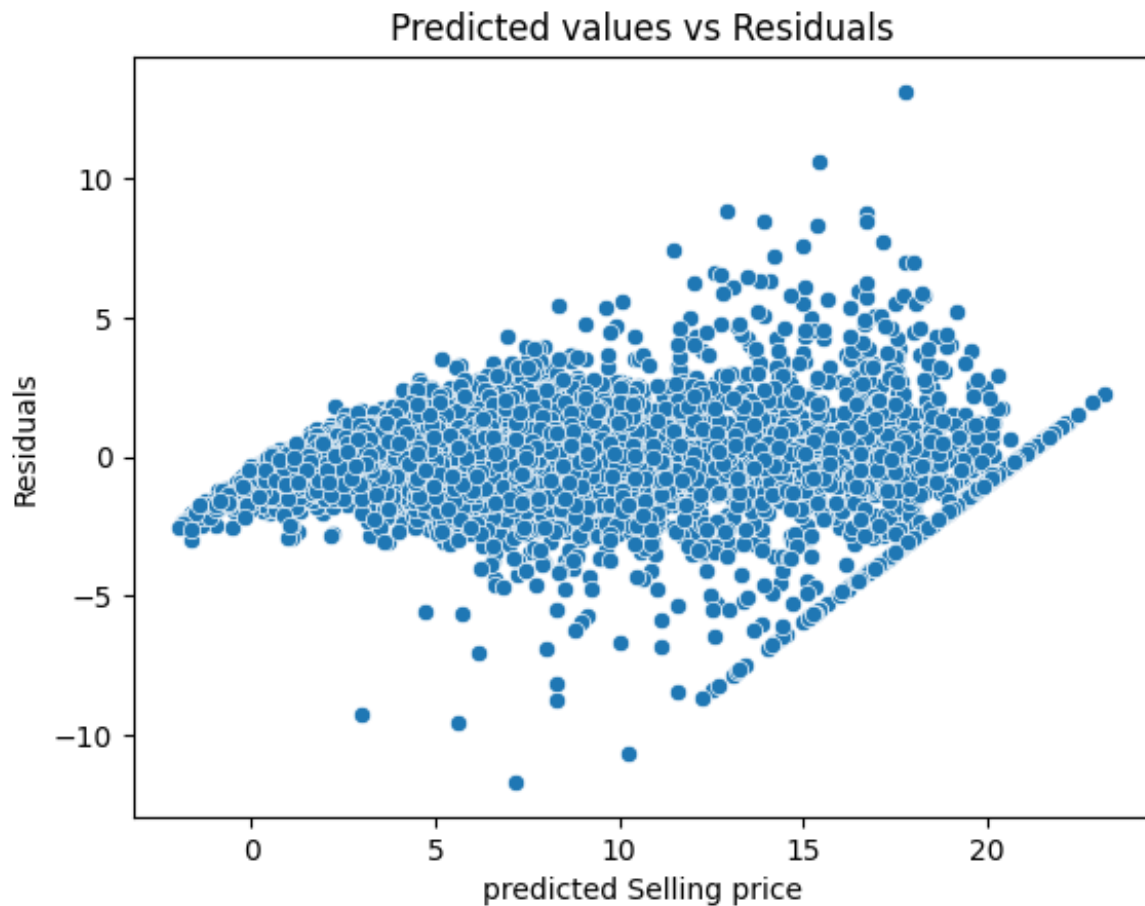
Code

```
sns.scatterplot(x=Y_hat,y=errors)
plt.xlabel("predicted Selling price")
plt.ylabel("Residuals")
plt.title("Predicted values vs Residuals")
```


Output

```
Text(0.5, 1.0, 'Predicted values vs Residuals')
```

Output Image



* Notice that As we go from left to right, the spread of errors is almost constant

What can we understand from this constant Residuals?

* We can assume that heteroskedasticity does not exist in our data

* There are outliers present in the dataset

We can also use Goldfeld-Quandt Test to verify our assumptions

Using Goldfeld Quandt Test to check homoskedacity

* This test is used to test the presence of Heteroscedasticity in the given data

* The Goldfeld-Quandt test works by removing some number of observations located in the center of the dataset, then testing to see if the spread of residuals is different from the resulting two datasets that are on either side of the central observations.

Code

```
# Performing the Goldfeld-Quandt test to check for Homoscedasticity -
from statsmodels.compat import lzip
import statsmodels.stats.api as sms
name = ['F statistic', 'p-value']
test = sms.het_goldfeldquandt(y_train, X2_sm)
lzip(name, test)
```

Output

```
[('F statistic', np.float64(1.0176613865185007)),
 ('p-value', np.float64(0.21807877686818042))]
```

From the goldfeld-quandt test:

- F Statistic comes out to be 1.00 => Implying minimal difference in variance between groups
- p-value of 0.353 indicates that this difference is statistically significant at conventional levels of significance (e.g., 0.05).

Therefore, we accept the null hypothesis of homoscedasticity, and conclude that there is no strong evidence of heteroscedasticity in the data.

Note:

In case we encounter heteroskedacity, we can use the following methods to remove the same:

- We can use transformations such as log, sq.rt., or boxcox transformation to remove heteroskedacity.
- Also, removing outliers will help.

Polynomial Regression

Say you are predicting sales of a product based on month data and the data looks like this:

Question: Will a regression line will be properly fit to the data ?

Ans: No. Itll not work

The equation for given curve will look somewhat like : $30 \sqrt{m} \cdot \cos(0.5m) + 5.83m + 78$

Lets take another example:

Say, you are given data of US Energy consumption vs year.

Question: Can you fit a regression line to it ?

No.

The equation of energy consumption may look like

$$c(\text{consumption}) = A + By + Dy^2 + Ey^3$$

* where y = year

* A, B, D and E are constants/coef.

Conclusion:

* As we can see, sometimes there can be variation in data

* Data may not look linear always

Recall the first assumption of Linear Regression?

There should be Linear relation between dependent and independent feature.

* But that assumption is being violated in this case.

What should be do ?

We take the help of polynomial regression in these cases.

Question: Suppose your data is following the following pattern. What plot is this ?

Ans: e^x

OR the data may look like this:

Lets take another example:

Say, you are given the following data (height, weight, gender, age) and youve to predict the Blood Pressure (BP)

Now, there's another aspect which may be important for calculating BP: BMI (Body Mass index)
- BMI will be a non-linear feature

Lets say we have multiple data points with raw attributes($x_1, x_2, x_3, x_4 \dots x_d$) and response variable (y)

Now, we create transformed features out of it (denoted using $\phi_i(x)$)

Question: What is the regression equation ?

Ans:

But since we are using transformed features, the equation would look like:

Code walkthrough

US electricity consumption based on year

Importing libraries

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn import metrics
```

Loading data

Code

```
!gdown 12wjTgfPAXwB7VNMVxm0LRBG8pjxG_ZqQ
```

Output

```
Downloading...  
From: https://drive.google.com/uc?id=12wjTgfPAXwB7VNMVxm0LRBG8pjxG_ZqQ  
To: /content/total-electricity-consumption-us.csv  
0% 0.00/680 [00:00<?, ?B/s]100% 680/680 [00:00<00:00, 3.08MB/s]
```

Code

```
#fetching data  
elec_cons = pd.read_csv("total-electricity-consumption-us.csv", sep = ',', header= 0 )  
elec_cons.head()
```

Output

	Year	Consumption
0	1920	57125
1	1921	53656
2	1922	61816
3	1923	72113
4	1924	76651

The dataset contains two variables - year and electricity consumption.

Total number of observations

Code

```
# number of observations: 51
elec_cons.shape
```

Output

```
(51, 2)
```

Are there any missing values?

Code

```
# checking NA
# there are no missing values in the dataset
elec_cons.isnull().values.any()
```

Output

```
np.False_
```

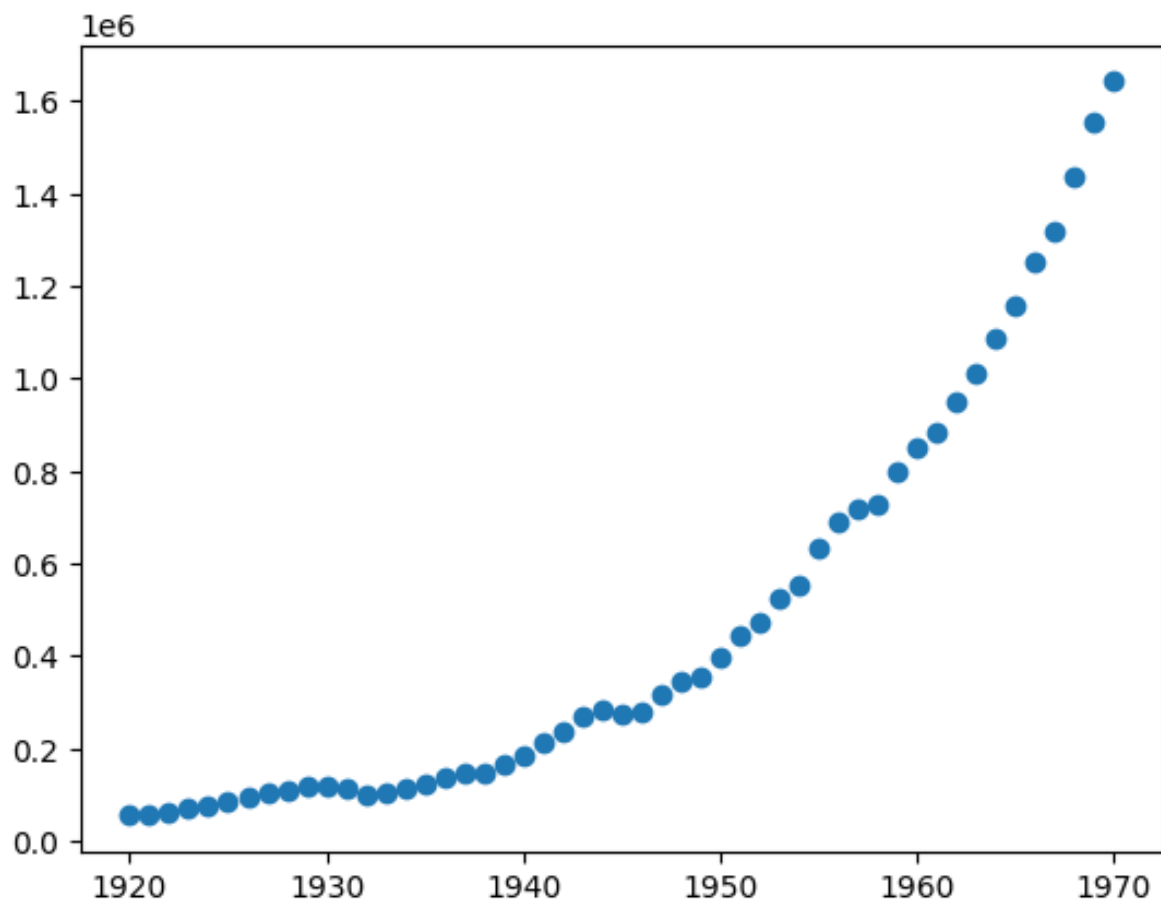
Code

```
plt.scatter(x=elec_cons['Year'], y= elec_cons['Consumption'])
```

Output

```
<matplotlib.collections.PathCollection at 0x78d87771bbf0>
```

Output Image



Splitting in train test

Since the number of datapoints are less, we are splitting it based on index for this exercise

Code

```
size = len(elec_cons.index)
index = range(0, size, 5)
train = elec_cons[~elec_cons.index.isin(index)]
test = elec_cons[elec_cons.index.isin(index)]
```

Code

```
print(len(train))  
print(len(test))
```

Output

```
40  
11
```

Reshaping

Code

```
# converting X to a two dimensional array, as required by the learning algorithm  
X_train = train.Year.values.reshape(-1,1) #Making X two dimensional  
y_train = train.Consumption  
X_test = test.Year.values.reshape(-1,1) #Making X two dimensional  
y_test = test.Consumption
```

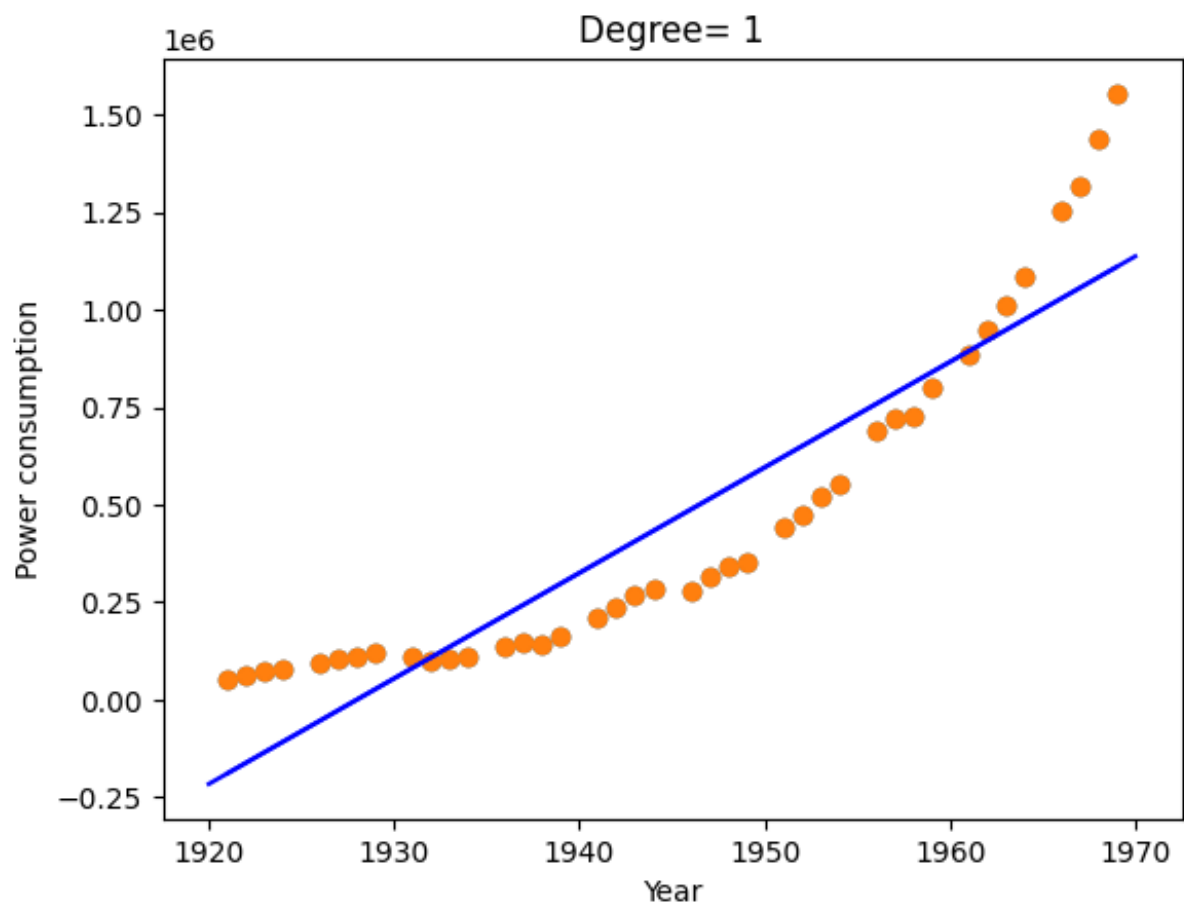
Fitting model

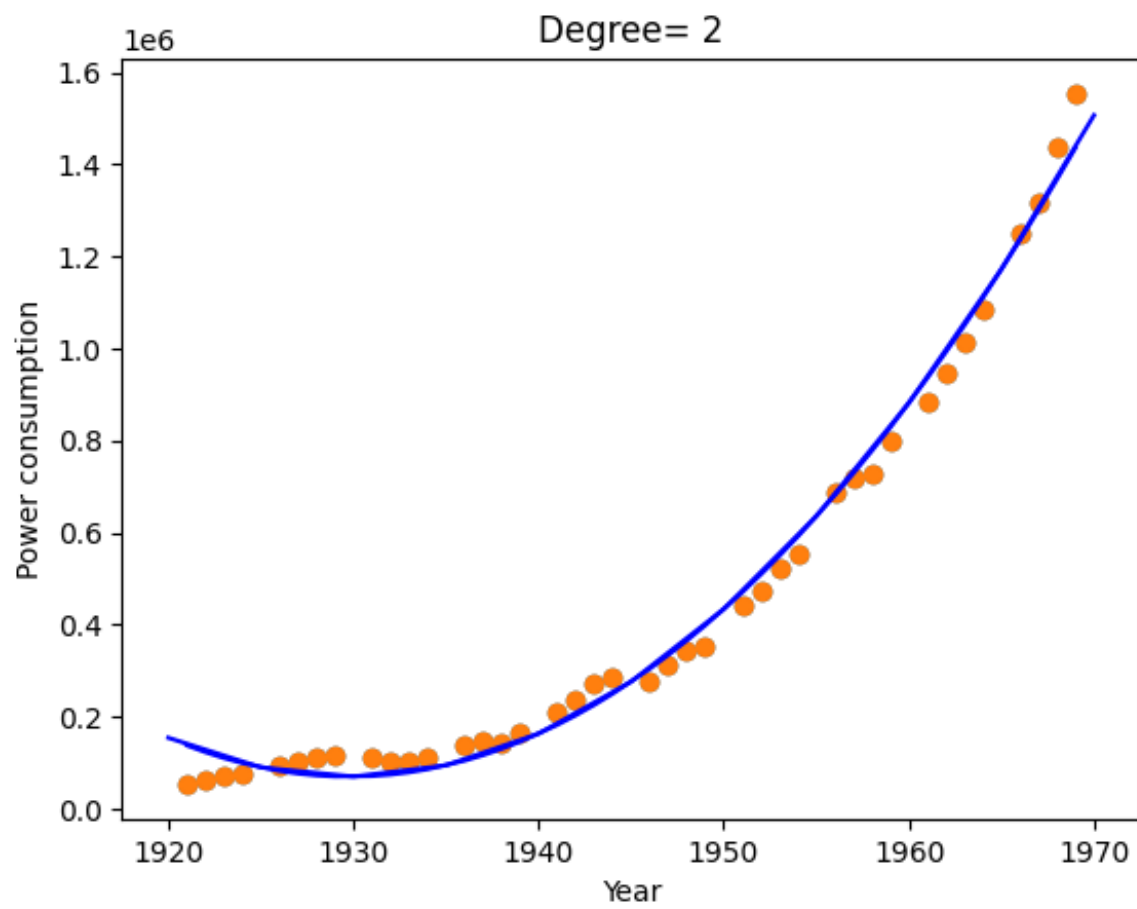
Code


```

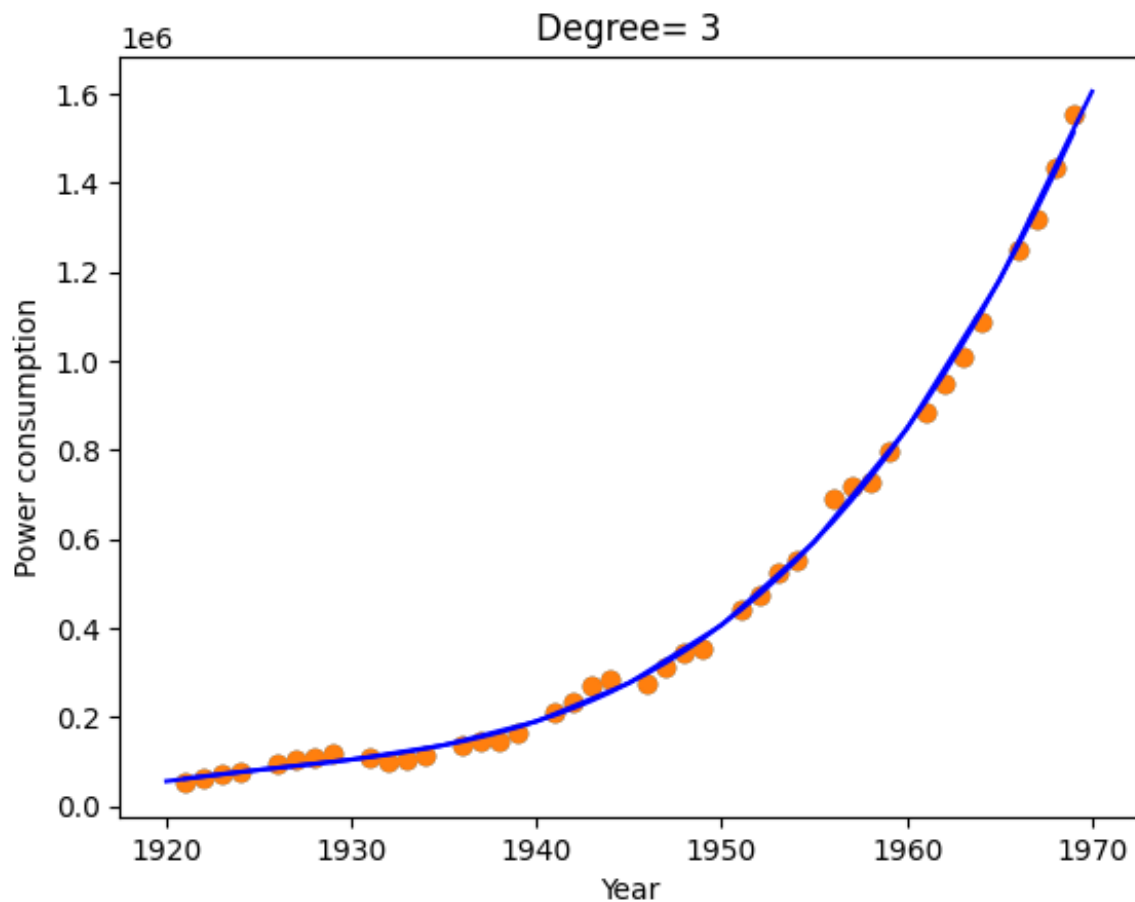
# Doing a polynomial regression: Comparing linear, quadratic and cubic fits
# Pipeline helps you associate two models or objects to be built sequentially with each other,
# in this case, the objects are PolynomialFeatures() and LinearRegression()
r2_train = []
r2_test = []
degrees = [1, 2, 3]
for degree in degrees:
    pipeline = Pipeline([('poly_features', PolynomialFeatures(degree=degree)),
                          ('model', LinearRegression())])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    r2_test.append(metrics.r2_score(y_test, y_pred))
    # training performance
    y_pred_train = pipeline.predict(X_train)
    r2_train.append(metrics.r2_score(y_train, y_pred_train))
    # plot predictions and actual values against year
    fig, ax = plt.subplots()
    ax.set_xlabel("Year")
    ax.set_ylabel("Power consumption")
    ax.set_title("Degree= " + str(degree))
    # train data in blue
    ax.scatter(X_train, y_train)
    ax.plot(X_train, y_pred_train, color="blue")
    # test data
    ax.scatter(X_train, y_train)
    ax.plot(X_test, y_pred, color="blue")
    plt.show()

```





Output Image



Code

```
# respective test r-squared scores of predictions
for i in degrees:
    print(f'For degree {i}, train R2 is {r2_train[i-1]} and test R2 is {r2_test[i-1]}')
```

Output

```
For degree 1, train R2 is 0.8423747402176137 and test R2 is 0.816517046382681
For degree 2, train R2 is 0.9908896744553399 and test R2 is 0.9876080502675171
For degree 3, train R2 is 0.9979789881922152 and test R2 is 0.9984897454348863
```

Note:

- Always check your R^2 on test data.

It may be possible that model is performing good on train data but not on test day (also called overfitting)

So, you should always Generalize

- * whatever you train a model, make sure you are testing it on unseen data.

- * and model performance on train and test should be similar

- * i.e. model is generalizing well on train as well as test data.

Occam's Razor

Say, youve two models.

First model: $y = ax_1 + c$

Second model: $y = ax_1^3 + bx_1^2 + cx_1 + d$

Performance of first model: 74%

Performance of second model: 78%

Question: Which model will you pick?

Occams razor says that pick the simplest model

So, well pick the first model.

- * Even if the other model (second one) is having slight improvement in terms of performance, we would like to avoid putting complex model into production.as they add less business value

Generally, the idea is to keep things as simple as one can.

Question: How do we decide at what degree should we stop to avoid overfitting?

Ans:

For the train data,

- as you include more degrees to your features, youll calculate R^2 for it

Similarly, youll calculate R^2 for Test data.

Suppose, you are increasing polynomials one by one and

- theres an increase in train performance

However, test performance didnt increase for second polynomial degree.

It is giving us an hint that the model is starting to overfit.

So, we keep a watch on test data

- it helps us understanding whether it is leading to overfitting.

Now, lets talk about overfitting and underfitting in detail

Bias Variance Tradeoff

Suppose youve data which looks like following:

Here,

- Green curve is overfitting as it is exactly matching the pattern of data
- Blue line is underfit as its not even trying to get near the pattern of data. Hence, the errors will be high.

Overfitting Scenario

Lets try to understand underfit overfit from a simple example:

Think of a student who has to appear for an exam.

- For preparation, s/he mugged up all the questions from text book.

In ML context, his R^2 over train data will be high (assume student to be model)

But, whatll be his/her performance on the exam ?

How will s/he do in the exam if exam doesnt consist of same question as textbook but a variety of those question?

- * Hence, performance or R^2 on test data will be low.

Underfitting Scenario

Similarly, in case of undefitting,

- * Suppose, this time student just only went through topics name and nothing else.

- * Obviously, preparation in this case is weak

- * i.e. performance on train data (R^2) is low

and henceforth, performance on test data will be low as well.

\$ \$

To summarize:

- Whenever we have an underfitting scenario,
- both train and test R^2 are low

- * In overfitting scenario,

train R^2 is highHowever, test R^2 is low

Now, we have to find the middle ground.

What will be perfect model ?

Perfect model will lie somewhere between these 2 scenarios of overfitting and underfitting

- * There will be error but these will be reasonable for both train and test
- * There'll a good balance across both train and test.

How do we do trade off between underfitting overfitting, mathematically?

Consider the model function

where $y_i = w_0 + w_1x_{i_1}^1 + w_2x_{i_1}^2 + w_3x_{i_1}^3 + w_4x_{i_1}^4$

- * f_1 or x_{i_1} is original feature
- * Rest of features are transformed features (power of f_1)

We have 3 cases

- Case 1 (Model M1): $w_1, w_2, w_3, w_4 \neq 0$
- Case 2 (Model M2): $w_1, w_2 \neq 0$ and $w_3 = w_4 = 0$
- Case 3 (Model M3): $w_2 = w_3 = w_4 = 0$ and $w_1 \neq 0$

Question: Which of these is more likely to be overfit model?

Model M1.

Here, Critical concept is weights

- * If weights are non zero ($w_1, w_2, w_3, w_4 \neq 0$), it means we are giving some importance to f_1, f_1^2, f_1^3 and f_1^4 .
- * Chances of having overfit model is more

Question: Similarly, which of the the above model is underfitting ?

Ans: Model M3

If weights on higher order polynomial $\neq 0$, more chances of overfit

Relating with Bias Variance

Lets try to understand with target shooting example:

High bias is equivalent to aiming in the wrong place.

High variance is equivalent to having an unsteady aim.

This can lead to the following scenarios:

- * Low bias, low variance: Aiming at the target and hitting it with good precision.
- * Low bias, high variance: Aiming at the target, but not hitting it consistently.
- * High bias, low variance: Aiming off the target, but being consistent.
- * High bias, high variance: Aiming off the target and being inconsistent.

Plotting R^2 for train test data

When we plot the R^2 for train and test vs the polynomial degree used in model

- * For train data, as the polynomial degree increases, the R^2 value increases and then stagnates.
- * For test data, the value of R^2 will increase at first and then itll fall off as polynomial degree is increase.

Question: Which side of the plot represents Overfit?

Ans: Right side as theres a high variance in train and test performance.

- Train R^2 is high and test R^2 is low.

Similarly, the leftmost portion of plot represents underfitting

- Both train and test R^2 are low.

Example of various model scenarios

- * Underfitting means that a model is too general, implying high bias - low variance (for example: fitting a line on quadratic data)
- * while overfitting means that a model is too specific, implying low bias - high variance. (for example: fitting higher order polynomial on quadratic data)

When training a model, it is important to balance these two. Since you cant realistically avoid bias and variance altogether, this is called the bias-variance tradeoff.

Regularization

Question: What is our loss in linear regression ?

Ans: Squared loss i.e. $(y - y?)^2$

Here, $y?$ can be represented using $w_1x_1 + w_2x_2 + w_0$

And the loss becomes: $(y_i - (w_1x_1 + w_2x_2 + w_0))^2$

Now, we want to minimize this loss as much as possible
- which means loss value should be small.

Question: When will the loss value be small ?

Loss will be small when the difference in actual y_i and predicted $y?_i$ is minimal.

So, in order to make the difference minimal, the model tries to make $y?_i$ values larger.

Question: How can we increase the value of $y?$?

By making coef/ weights (w_0, w_1, w_2) larger.

In the process of minimizing error, the coef./ weights becomes very large value.
- and it becomes a challenge to handle.

How to handle it ?

The issue is large values of w_i . So, what if we add w_i^2 in our loss.

* When we try to increase values of coef./weights (w_0, w_1, w_2) in order to minimize the loss, the second term (w_i^2) value increases which increases the overall value of loss.

Basically, it is a kind of penalty.

Example

Imagine you have a problem of roof leakage in your house.

How would you solve it ?

We have 3 remedies to stop the leakage

1. Bandage (Leakage - 1000 mL)
2. Shingles (Leakage - 10 mL)
3. Titanium (Leakage - 0 mL)

Say, every solution has a cost attached to it

1. Bandage (Cost - Rs 1)
2. Shingles (Cost - Rs 1000)
3. Titanium (Cost - Rs 100,000)

Observe the tug of war between Leakage and Budget.

- * If you reduce the leakage with titanium tiles, cost shoots up
- * If you use bandage for leakage, cost is in control but leakage doesn't reduce

What do we have to keep in mind for making decision?

- * We want to minimize the leakage
- * We want to minimize the budget as well.

i.e. $\text{minimize (Leakage + Budget)}$

Here, consider

- Leakage as your Loss(MSE) and
- Budget as a new term called Regularization.

We want to $\text{minimize } \text{space (LOSS + REG.)}$

There can be case that when penalization becomes too high

- then all the coef. will be a small value.
- model will perform bad.

So, we need a control over regularization

But, how do we control the regularization ?

* We introduce a term λ ($\lambda > 0$) which is multiplied with the regularization term ($\sum_j w_j^2$).

* The optimization equation becomes

The regularization term is still going to add to the loss but we will control how much it'll add to it.

Penalization should happen. But in a controlled way.

How does value of lambda impact the model?

Optimization eq. = Loss term + λ Regularization term

* Case1: $\lambda = 0$

There will be no regularization term. Model will overfit.

* Case2: $\lambda = \lambda$ (large +ve value)

As value of lambda is large, the magnitude of squared loss term is negligible. Weightage of regularization term will increase. It'll try to push all the w_j s to zero. Model will underfit.

Lambda, here is called hyperparameter of the model.

Hyperparameters help us do overfit- underfit tradeoff. By changing the lambda, we can control the regularization and hence overfit and underfit of the model.

As lambda increase, we tend from overfitting to underfitting.

Note:

When we define a model,

- the weights of the model become our parameters
- And the rest of the variables such as Regularization parameter become our Hyperparameter

Types of Regularization

L2 Regularization

Here, w_j^2 term is called as L2 regularization.

- We also call it as Ridge Regression

L1 Regularization

Instead of taking square of w_i , if we use the modulus i.e. $|w_i|$

- then it is called as L1 regularization
- or Lasso Regression

Key highlight of Lasso Regression is:

- it is very good in giving sparse data
- i.e. whatever columns are irrelevant, it'll make its weights/ coef as 0.

Elastic net regularization

* Combination of L1 and L2 regularization

We'll go through the code walkthrough of it in the next class